

KarPool

Smart University Carpool Management Platform

Software Requirements Specification (SRS)

Prepared by: Campus Carpool Crew
Shayan Raza | Asad Siddiqui | Sami Ul Hassan

Date: 22nd February 2026
Course: Software Engineering

1. Introduction & Project Overview

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for KarPool — a centralized web-based carpool management platform designed specifically for university students. This document is intended for the development team, academic supervisors, and all stakeholders involved in the design, development, and evaluation of the system.

1.2 Problem Statement

University students in Karachi face daily commuting challenges including high transportation costs, severe traffic congestion, safety concerns, and a complete absence of organized ride-sharing systems. Currently, students rely on informal WhatsApp groups or word-of-mouth coordination, which lack structure, verification, and proper seat management. The absence of a centralized digital platform results in:

- Difficulty finding reliable, verified carpool partners
- Poor communication between drivers and riders
- Safety risks due to unverified participants
- Inefficient seat management with no tracking
- No ride history or accountability mechanism

1.3 Project Purpose

KarPool aims to solve the above challenges by providing a structured, secure, and user-friendly platform that connects university students as drivers and riders within the same university ecosystem.

1.4 Scope

KarPool is a web-based platform accessible via browser and a mobile wrapper (React Native/Expo). The platform will serve students of one or more universities in Karachi. It will support user registration, role-based access (Driver/Rider), ride offering and booking, seat management, communication via WhatsApp integration, and an admin monitoring panel. The system will be developed using an Agile-Scrum methodology.

2. Functional Requirements

All functional requirements are uniquely numbered using the format FR-XX. Each requirement is testable and mapped to a priority level (High, Medium, Low).

2.1 Authentication & User Management

ID	Requirement Description	Priority	Rationale
FR-01	The system shall allow new users to register using their university email address.	High	Ensures only verified university students can access the platform.
FR-02	The system shall authenticate users via email/password or SSO (university portal).	High	Provides secure, institution-verified login.
FR-03	The system shall allow users to select a role (Driver or Rider) during registration.	High	Role-based access is fundamental to the carpool model.
FR-04	The system shall allow users to update their profile information including name, phone number, and profile photo.	Medium	Enables riders to identify drivers and vice versa for trust.
FR-05	The system shall allow users to switch roles between Driver and Rider from their profile settings.	Medium	Provides flexibility for students who both drive and ride at different times.
FR-06	The system shall allow users to reset their password via an email-based OTP verification process.	High	Ensures account recovery without compromising security.

2.2 Offer a Ride (Driver)

ID	Requirement Description	Priority	Rationale
FR-07	The system shall allow a Driver to create a ride offer by specifying origin area, date, departure time, and number of available seats. The destination university and WhatsApp contact number shall be automatically inherited from the driver's registered profile.	High	Core functionality — drivers must be able to post rides. Destination and contact are automated from the driver's profile.
FR-08	The system shall allow a Driver to set a cost-sharing amount (PKR) per seat when creating a ride offer.	Medium	Supports cost-sharing model to reduce commute expenses.
FR-09	The system shall allow a Driver to view all their active and past ride offers from a dashboard.	High	Enables ride management and history tracking for drivers.

ID	Requirement Description	Priority	Rationale
FR-10	The system shall allow a Driver to cancel or edit an active ride offer before any seats are booked.	High	Provides flexibility for schedule changes.
FR-11	The system shall notify the Driver via in-app notification when a Rider books a seat in their ride.	High	Ensures drivers are informed of confirmed bookings immediately.
FR-11A	The system shall allow a Driver to mark an active ride as 'Completed' once the journey is finished.	High	Necessary to unlock the review and rating system for passengers.

2.3 Find & Book a Ride (Rider)

ID	Requirement Description	Priority	Rationale
FR-12	The system shall allow a Rider to search for available rides by entering origin, destination, and preferred date.	High	Core functionality — riders must be able to find suitable rides.
FR-13	The system shall display a list of matching ride offers with driver details, departure time, available seats, and cost per seat.	High	Provides riders with sufficient information to make a booking decision.
FR-14	The system shall allow a Rider to book one or more available seats in a ride offer.	High	Core booking functionality.
FR-15	The system shall automatically decrement the available seat count when a Rider books a seat.	High	Prevents overbooking and maintains seat availability accuracy.
FR-16	The system shall prevent a Rider from booking a ride that has zero available seats.	High	Ensures data integrity and correct seat management.
FR-17	The system shall send a booking confirmation notification to the Rider upon successful seat reservation.	High	Provides immediate confirmation feedback to the rider.
FR-18	The system shall allow a Rider to cancel their booking up to 1 hour before the departure time.	Medium	Provides flexibility while maintaining fairness to drivers.

2.4 Communication

ID	Requirement Description	Priority	Rationale
FR-19	The system shall provide a WhatsApp deep-link button on each ride listing, allowing a Rider to contact the Driver directly via WhatsApp.	High	Provides a familiar, reliable communication channel for Pakistani students.
FR-20	The system shall display the Driver's contact number on the booking confirmation page, visible only to confirmed Riders.	Medium	Protects driver privacy while enabling confirmed riders to coordinate.

2.5 Smart Location Search

ID	Requirement Description	Priority	Rationale
FR-21	The system shall integrate a location autocomplete API to suggest address completions when a user types an origin or destination.	High	Reduces input errors and speeds up the ride search/create process.
FR-22	The system shall allow users to select from pre-defined university campus locations as valid destinations.	Medium	Simplifies destination entry for the most common use case.

2.6 Ride History

ID	Requirement Description	Priority	Rationale
FR-23	The system shall maintain and display a ride history for both Drivers (rides offered) and Riders (rides booked), showing date, route, and status.	High	Enables accountability and review of past activity.
FR-24	The system shall mark rides as 'Completed' automatically once the departure time has passed.	Medium	Automates status management without manual intervention.

2.7 Admin Monitoring Panel

ID	Requirement Description	Priority	Rationale
FR-25	The system shall provide an Admin dashboard displaying total registered users, active ride offers, and completed rides.	High	Enables system oversight and monitoring of platform activity.

ID	Requirement Description	Priority	Rationale
FR-26	The system shall allow an Admin to deactivate a user account in cases of policy violation.	High	Provides platform moderation capability.
FR-27	The system shall allow an Admin to view all ride listings and booking records for audit purposes.	Medium	Enables traceability and accountability across the platform.

2.8 Ratings & Reviews

Since a comprehensive rating system has been built, it requires its own subsection under Functional Requirements.

ID	Requirement Description	Priority	Rationale
FR-28	The system shall allow a Rider to submit a star rating (1–5) and an optional text review for a Driver, but only after the Driver has marked the ride as 'Completed'.	High	Ensures reviews are only submitted for completed journeys, maintaining data integrity.
FR-29	The system shall dynamically calculate a Driver's average rating and total review count, and display these metrics on the Driver's profile and on all their active ride listings in the search results.	High	Provides riders with reliable social proof when choosing a driver.

3. Non-Functional Requirements

Non-functional requirements define the quality attributes of KarPool. They are categorized and include measurable criteria.

3.1 Performance

ID	Requirement	Measurable Criterion
NFR-P1	The system shall load the homepage and ride listing page within an acceptable response time.	Page load time < 3 seconds on a standard 4G connection.
NFR-P2	The system shall handle concurrent users without performance degradation.	System supports at least 200 simultaneous users with < 5% error rate.
NFR-P3	API endpoints shall respond within an acceptable latency window.	95th-percentile REST API response time < 500 ms under normal load.

3.2 Security

ID	Requirement	Measurable Criterion
NFR-S1	All data transmitted between client and server shall be encrypted.	HTTPS (TLS 1.2+) enforced for all endpoints; no HTTP allowed.
NFR-S2	User passwords shall be stored securely and never in plain text.	Passwords hashed using bcrypt with a minimum cost factor of 12.
NFR-S3	API access shall be protected using token-based authentication.	JWT tokens issued on login with a maximum expiry of 24 hours.
NFR-S4	The system shall restrict access to resources based on user roles.	Unauthorized role-based access attempts return HTTP 403; verified by penetration test.

3.3 Usability

ID	Requirement	Measurable Criterion
NFR-U1	The interface shall be intuitive enough for a new user to complete a ride booking without guidance.	A new user can complete a ride booking in <= 5 clicks during usability testing.
NFR-U2	The platform shall be responsive and function correctly on mobile browsers.	All pages render correctly on screen widths from 360px (mobile) to 1440px (desktop).
NFR-U3	The system shall display clear error messages for invalid inputs.	100% of form validation errors display a descriptive message within 1 second.

3.4 Reliability & Availability

ID	Requirement	Measurable Criterion
NFR-R1	The platform shall maintain high uptime during university operating hours (6 AM – 10 PM).	System uptime $\geq$ 99% during university operating hours (measured monthly).
NFR-R2	The system shall back up database records at regular intervals.	Automated database backups run every 24 hours with a 7-day retention policy.

3.5 Maintainability

ID	Requirement	Measurable Criterion
NFR-M1	The codebase shall follow consistent coding standards to facilitate future maintenance.	Code linted with ESLint/Black; all modules documented with inline comments.

4. Use Cases / User Stories

4.1 Use Case: Register & Login

Use Case ID	UC-01
Name	Register & Login
Actor(s)	New User (Driver or Rider)
Precondition	User has a valid university email address.
Main Flow	1. User navigates to the registration page. 2. User enters university email, password, full name, WhatsApp number, and selects their University. 3. System validates the email domain against allowed university domains. 4. System creates the account and sends a verification email. 5. User verifies their email and logs in with credentials. 6. System issues a JWT token and redirects to the dashboard.
Alternate Flow	A1. Email is not a valid university domain: System shows error 'Please use your university email'. A2. Email already registered: System prompts user to log in or reset password.
Postcondition	User has a verified account and is authenticated in the system.

4.2 Use Case: Offer a Ride

Use Case ID	UC-02
Name	Offer a Ride
Actor(s)	Driver
Precondition	Driver is logged in with the Driver role.
Main Flow	7. Driver clicks 'Offer a Ride' from the dashboard. 8. Driver enters origin, departure date, time, and available seats. The system automatically populates the destination with the driver's registered university and attaches their registered WhatsApp number. 9. Driver submits the form. 10. System validates all fields and saves the ride offer with status 'Active'. 11. Ride appears in the Driver's 'My Rides' dashboard and is visible to Riders in search results.
Alternate Flow	A1. Departure time is in the past: System shows error 'Departure time must be in the future'. A2. Seats field is 0: System shows validation error.
Postcondition	An active ride offer is saved and visible to all Riders in search results.

4.3 Use Case: Find & Book a Ride

Use Case ID	UC-03
Name	Find & Book a Ride
Actor(s)	Rider
Precondition	Rider is logged in. At least one active ride offer exists.
Main Flow	12. Rider navigates to 'Find a Ride' and enters origin, destination, and date. 13. System displays matching ride offers with driver name, departure time, available seats, and cost. 14. Rider selects a ride and clicks 'Book Seat'. 15. System confirms the booking, decrements available seats by 1, and sends a confirmation notification. 16. Rider sees the Driver's WhatsApp contact for further coordination.
Alternate Flow	A1. No rides match the search: System shows 'No rides found for your criteria'. A2. Rider attempts to book a fully-booked ride: System shows 'No seats available' and disables the Book button.
Postcondition	Rider has a confirmed booking. Available seats on the ride are decremented by the booked amount.

4.4 Use Case: Rate & Review Driver

Use Case ID	UC-04
Name	Rate & Review Driver
Actor(s)	Rider
Precondition	Rider has an accepted booking and the Driver has marked the ride as 'Completed'.
Main Flow	17. Rider views past journeys. 18. Rider clicks "Leave Review". 19. Rider provides a 1–5 star rating and optional comment. 20. System updates the Driver's average rating and total review count.
Alternate Flow	A1. Driver has not yet marked the ride as Completed: 'Leave Review' button is disabled.
Postcondition	Driver's average rating and review count are updated and reflected across the platform.

5. Out of Scope

The following features are explicitly excluded from the KarPool system in its current version to maintain focus and feasibility within the project timeline:

Feature Excluded	Reason / Future Consideration
Direct online payment processing (e.g., JazzCash, EasyPaisa)	Requires payment gateway integration and compliance.
AI-based ride matching algorithms	Requires large training datasets. Manual search is sufficient for initial launch.
Integration with government vehicle databases (MTMIS)	Requires official API access and regulatory approval.
Advanced GPS real-time ride tracking	Requires real-time infrastructure (WebSockets + GPS).
Native mobile application (iOS / Android)	A React Native wrapper (Expo) provides mobile access.
In-app chat / messaging system	WhatsApp deep-link serves this purpose.

6. Assumptions, Constraints & Dependencies

6.1 Assumptions

- All users possess a valid, active university email address.
- Users have access to a smartphone or desktop browser with an internet connection.
- Drivers will self-declare their vehicle capacity accurately; no automated vehicle verification is performed.
- WhatsApp is installed and in use by the target student population, making it a viable communication channel.
- The university has a consistent email domain that can be used for registration validation.

6.2 Constraints

- Development must be completed within one academic semester using an Agile-Scrum methodology managed on Trello.
- The platform will be developed and tested in a limited environment without access to production-level cloud infrastructure.
- No payment processing is included due to regulatory and financial constraints.
- Real-time GPS tracking is excluded due to infrastructure complexity.

6.3 Dependencies

Dependency	Type	Details
Next.js	Frontend Framework	React-based SSR framework with Tailwind CSS
FastAPI	Backend Framework	Python-based REST API framework
PostgreSQL/SQLite	Database	Primary data store for users, rides, and bookings
WhatsApp API	External Integration	Deep-link for driver-rider communication
GitHub	Version Control	Source code management and collaboration
Trello	Project Management	Agile backlog and sprint management per instructor structure

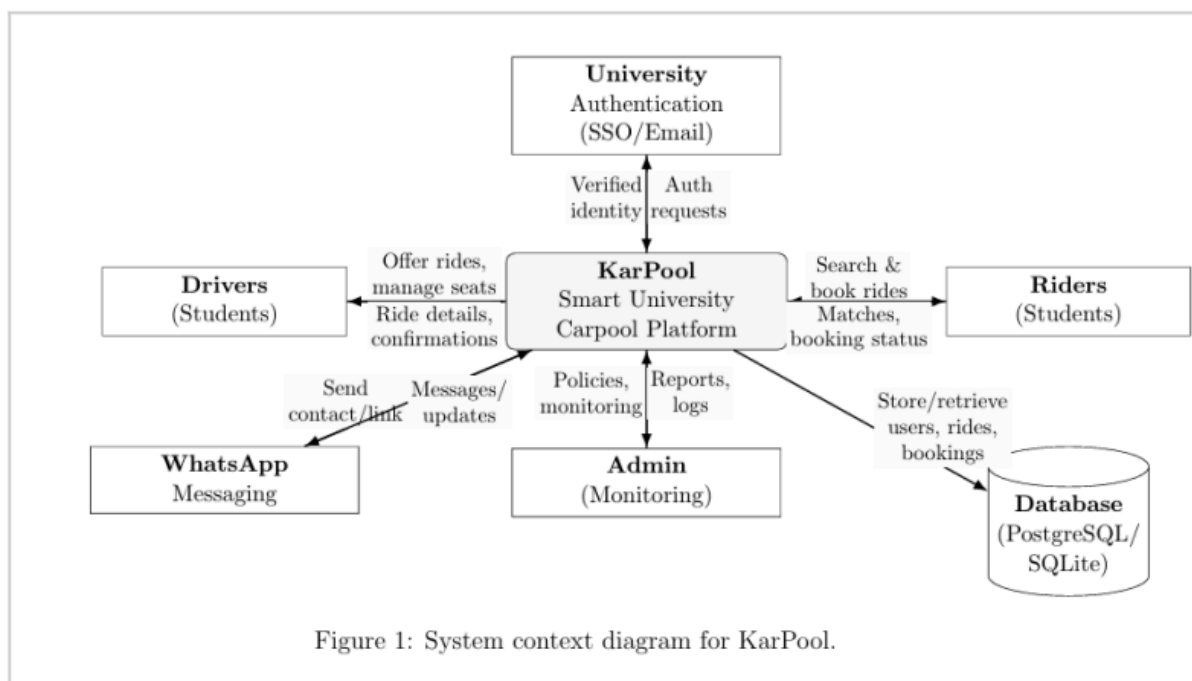
7. Diagrams & Supporting Models

7.1 System Context Diagram

The following system context diagram (from the submitted project proposal) illustrates the high-level interactions between KarPool and its external actors:

Key actors and their interactions:

- Drivers (Students): Offer rides, manage seats, and receive booking notifications.
- Riders (Students): Search and book rides, receive confirmations, and contact drivers via WhatsApp.
- University Authentication: Validates user identities via SSO/Email during registration and login.
- Admin (Monitoring): Reviews platform activity, manages users, and accesses reports.
- WhatsApp Messaging: Facilitates direct communication between confirmed drivers and riders.
- Database (PostgreSQL/SQLite): Stores and retrieves all user, ride, and booking data.



7.2 Use Case Diagram

The following actors and use cases are identified for KarPool:

Actor	Associated Use Cases
Unregistered User	Register Account, Login
Driver	Login, Offer Ride, View My Rides, Edit/Cancel Ride Offer, View Ride History, Update Profile, Mark Ride as Completed

Actor	Associated Use Cases
Rider	Login, Search Ride, Book Seat, Cancel Booking, Contact Driver via WhatsApp, View Ride History, Update Profile, Rate & Review Driver
Admin	Login, View Dashboard, View All Rides, View All Bookings, Deactivate User

